

Sorting Algorithms: Summary & Comparison

Algorithm Summaries

1. Bubble Sort

How it works: Repeatedly compares adjacent elements and swaps them if in wrong order. Includes optimization with swapped flag to exit early.

Best for: Small datasets, educational purposes

Trade-off: Simple but inefficient for large data

2. Insertion Sort

How it works: Builds sorted array by inserting each element into correct position in already-sorted portion.

Best for: Small datasets, nearly-sorted data

Trade-off: Efficient for small inputs but $O(n^2)$ for large unsorted

3. Selection Sort

How it works: Finds minimum element and places it at beginning. Makes exactly $n-1$ passes regardless of input.

Best for: Memory writes are expensive

Trade-off: Consistent $O(n^2)$ but no early exit

4. Merge Sort

How it works: Divide-and-conquer: recursively splits array, sorts each half, then merges back together.

Best for: Large datasets, consistent performance

Trade-off: Requires $O(n)$ extra space

5. Quick Sort

How it works: Partitions array using pivot, recursively sorts sub-arrays (Lomuto partition scheme).

Best for: Large datasets, general-purpose

Trade-off: $O(n^2)$ worst case with poor pivot choices

Time Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✗

When to Use Each Algorithm

Scenario	Best Choice
Small datasets (<50 elements)	Insertion Sort or Bubble Sort
Already nearly-sorted data	Insertion Sort
Large datasets	Merge Sort or Quick Sort
Need guaranteed $O(n \log n)$	Merge Sort
Limited memory	Quick Sort
Data stream/real-time	Insertion Sort
Educational/learning	Bubble Sort or Selection Sort